



oui fait le

Voici une checklist "prod RAG" orientée rapidité pour une stack FastAPI + Postgres + Chroma + Ollama, avec valeurs par défaut raisonnables (à adapter à ta machine et à ton volume).

1. Configuration générale

- Objectif de latence (end-to-end, P95)
 - 1ère token côté front : 300–800 ms.
 - Réponse complète : 1–3 s pour 2–4 paragraphes.
- Logs par étape (avec un `request_id`)
 - `t_retrieval_ms`
 - `t_rerank_ms` (si tu rerank)
 - `t_llm_first_token_ms`
 - `t_llm_total_ms`
 - `prompt_tokens`, `completion_tokens`.

2. Ingestion et chunking

- Pré-traitement documents
 - Supprimer menus, footers, contenus dupliqués.
 - Normaliser les titres, sections (H1/H2), dates, tags.
- Paramètres de chunking
 - Taille cible : 400–700 tokens (ou ~800–1500 caractères) par chunk.
 - Overlap : 10–15% du chunk (ex. 50–80 tokens).
 - Chunk sémantique (par paragraphe/section logique, pas coupé au milieu d'une phrase).
- Métadonnées utiles par chunk
 - `source_id`, `type` (doc, faq, ticket, code, etc.), `lang`, `created_at`, `updated_at`, `tags`.

3. Embeddings et index Chroma

- Modèle d'embeddings
 - Un modèle rapide (type `all-MiniLM`, ou équivalent) plutôt que très gros.
 - Dimension typique : 384–768.
- Stratégie d'index

- Index HNSW ou équivalent ANN.
- Param par défaut (à affiner) :
 - `M`: 16–32
 - `ef_construction`: 100–200
 - `ef_search`: 32–64
- Batch d'indexation
 - Ingestion en batch (par ex. 256–1024 chunks) pour limiter les allers-retours.
 - Jobs d'ingestion asynchrones (pas bloquer le trafic utilisateur).

4. Retrieval (FastAPI → Chroma)

- Paramètres de recherche
 - `k initial` : 5 (monter à 8–10 si nécessaire, mais pas 50).
 - Filtrage par métadonnées (ex. scope de l'utilisateur, langue, type de doc) directement dans la query.
- Hybrid search (si pertinent)
 - Combiner dense + BM25 quand tu as beaucoup de texte "classique" (docs, FAQ).
 - Fusion par RRF ou simple re-score.
- Latence cible
 - `t_retrieval P95` < 120–150 ms.

5. Rerank (optionnel mais conseillé à volume moyen/élevé)

- Rerank léger
 - Petits modèles de rerank (cross-encoder) ou scoring LLM très court.
 - Appliqué sur les top 20 résultats, pour en garder 5–8.
- Latence cible
 - `P95` < 100 ms (sinon le désactiver sur les petites requêtes / mode "fast").

6. Construction du prompt

- Gabarit minimaliste
 - System : rôle + 3–5 règles max.
 - Contexte : 3–8 passages, chacun précédé d'un header (titre, source, date).
 - Question : brute, sans reformulation verbeuse.
- Longueur max
 - Contexte total : idéalement < 2000–2500 tokens.
 - Prompt complet (system + instructions + contexte + question) à monitorer, viser < 3000–3500 tokens en moyenne.

- Stratégies avancées
 - Si contexte > limite :
 - soit résumer une partie avant injection,
 - soit faire retrieval hiérarchique (trouver la bonne section, puis re-retrieval dedans).

7. LLM (Ollama) et génération

- Choix du modèle
 - Modèle de base : 7–8B, quantifié (Q4/Q5 ou FP16 si tu as un bon GPU).
 - Température basse pour RAG (0.0–0.3) pour limiter la variabilité et la longueur.
- Param génération (valeurs de départ)
 - `max_tokens` : 512–1024 pour un chat "classique".
 - `top_p` : 0.9–0.95
 - `presence_penalty`, `frequency_penalty` proches de 0 pour éviter les déviations.
- Streaming
 - Activer le streaming des tokens vers FastAPI, puis vers le front (SSE/WebSocket).
 - Côté UI, afficher dès le 1er chunk, même incomplet.

8. Architecture FastAPI / Postgres / Chroma / Ollama

- Réseau et déploiement
 - Tous les services dans le même réseau local ou même VPC/zone.
 - Keep-alive et pooling HTTP pour parler à Ollama/Chroma (pas de reconnect à chaque requête).
- Concurrency
 - FastAPI en async + pool de workers raisonnable (gunicorn/uvicorn).
 - Queue interne si tu as un GPU partagé (ne pas saturer le modèle avec trop de requêtes simultanées).
- Postgres
 - Utilisé pour : logs, sessions, profils, stats RAG.
 - Ne pas faire de grosses requêtes SQL synchrones sur le chemin critique de la réponse.

9. Caches et "speed modes"

- Caches recommandés
 - Cache de résultats RAG complets pour les requêtes les plus fréquentes (en base ou Redis).
 - Cache de retrieval (query → ids de chunks) pour les FAQ/intent récurrentes.
 - Cache des documents "pinned" en mémoire (top N chunks les plus utilisés).

- Mode "fast" vs "full"
 - Fast :
 - k faible (3–5), pas de rerank, modèle plus petit ou quantifié plus fort, context court.
 - Full :
 - k un peu plus élevé, rerank activé, modèle principal, plus de contexte.

10. Observabilité et tuning continu

À mettre en place dès le début :

- Métriques
 - Latence P50/P95 par étape (retrieval, rerank, LLM).
 - Taille moyenne des prompts, context_tokens, answer_tokens.
 - Taux d'erreur LLM / Chroma / DB.
 - Hit-rate des caches (RAG complet, retrieval, docs).
- Traces
 - Pour chaque requête : question, ids de chunks utilisés, scores, longueur de la réponse.
 - Corrélation avec feedback utilisateur (like/dislike, rating).
- Boucle de tuning
 - Si latence trop haute :
 - réduire k, chunk size, ou la longueur du prompt,
 - alléger le modèle (quantization, modèle plus petit),
 - ajuster ef_search/param ANN.
 - Si qualité insuffisante :
 - revoir chunking (plus sémantique),
 - augmenter légèrement k,
 - activer rerank,
 - spécialiser le modèle ou adapter le prompt.

Si tu veux, tu peux m'envoyer la description de ton endpoint FastAPI actuel (schéma d'input/output + ce que tu logues), et je te propose une version "optimisée latence" avec la checklist intégrée directement dans le pipeline.